

ReflectionPad2d#

<https://pytorch.org/docs/stable/generated/torch.nn.ReflectionPad2d.html>

Description:

Pads the input tensor using the reflection of the input boundary. For N-dimensional padding, use `torch.nn.functional.pad()`. padding (int, tuple) – the size of the padding. If is int, uses the same padding in all boundaries. If a 4-tuple, uses `(padding_left\text{padding\_left}padding_left, padding_right\text{padding\_right}padding_right, padding_top\text{padding\_top}padding_top, padding_bottom\text{padding\_bottom}padding_bottom)` Note that padding size should be less than the corresponding input dimension. Input: `(N,C,Hin,Win)``(N, C, H_{in}, W_{in})``(N,C,Hin,Win)` or `(C,Hin,Win)``(C, H_{in}, W_{in})``(C,Hin,Win)`. Output: `(N,C,Hout,Wout)``(N, C, H_{out}, W_{out})``(N,C,Hout,Wout)` or `(C,Hout,Wout)``(C, H_{out}, W_{out})``(C,Hout,Wout)` where

Function Signature

```
class torch.nn.ReflectionPad2d(padding)[source]#
```

Parameters

Shape:

Code Examples

```
>>> m = nn.ReflectionPad2d(2)
>>> input = torch.arange(9, dtype=torch.float).reshape(1, 1, 3,
3)
>>> input
tensor([[[[0., 1., 2.],
          [3., 4., 5.],
          [6., 7., 8.]]]]])
>>> m(input)
tensor([[[[8., 7., 6., 7., 8., 7., 6.],
          [5., 4., 3., 4., 5., 4., 3.],
          [2., 1., 0., 1., 2., 1., 0.],
          [5., 4., 3., 4., 5., 4., 3.],
          [8., 7., 6., 7., 8., 7., 6.],
          [5., 4., 3., 4., 5., 4., 3.],
          [2., 1., 0., 1., 2., 1., 0.]]]]]])
>>> # using different paddings for different sides
>>> m = nn.ReflectionPad2d((1, 1, 2, 0))
>>> m(input)
tensor([[[[7., 6., 7., 8., 7.],
          [4., 3., 4., 5., 4.],
          [1., 0., 1., 2., 1.],
          [4., 3., 4., 5., 4.],
          [7., 6., 7., 8., 7.]]]]]])
```

Detailed Documentation

Documentation

Pads the input tensor using the reflection of the input boundary.

For N-dimensional padding, use `torch.nn.functional.pad()`.

padding (int, tuple) – the size of the padding. If is int, uses the same padding in all boundaries. If a 4-tuple, uses $(padding_left, padding_right, padding_top, padding_bottom)$. Note that padding size should be less than the corresponding input dimension.

Input: (N, C, H_{in}, W_{in}) or (C, H_{in}, W_{in})

Output: (N, C, H_{out}, W_{out}) or (C, H_{out}, W_{out}) where

$H_{out} = H_{in} + padding_top + padding_bottom$

$W_{out} = W_{in} + padding_left + padding_right$